

JP's VPX Physics 2026

(tested on VPX8 and VPX8.1 beta)

(This is a short introduction to my physics settings for authors who want to try them. So they are not a plug and play for players, but for authors or modders)

What are there new physics settings?

These physics set is just an expanded and more complete set of the default VPX physics. It includes settings for materials, like plastics, metals, rubbers and also for VPX objects like slingshots, flippers, and the table itself. It also includes a few short scripts.

You will find several files in the download zip:

- **JP's VPX Physics 2026.pdf**

The physical materials for easily change the object properties:

- **JP's VPX Physics 2026 EM Materials.mat**
- **JP's VPX Physics 2026 SS Materials.mat**

The physical proterties of the table and flippers:

- **JP's VPX Physics 2026 EM Table physics.vpp**
- **JP's VPX Physics 2026 SS Table physics.vpp**

Short explanation about the physics in a VPX table

How a VPX table behaves depends on several things: the first is the playfield properties, then on all the object's properties the ball collides with, those can be walls, targets, primitives, ramps and also the properties of the built-in VPX objects, like bumpers, slingshots and flippers.

I have divided my settings in two grups: EM and SS settings. The EM settings are for older tables, which usually were slower and more bouncy than modern tables. So EM settings for older tables, SS for newer tables, mostly the ones with ramps.

How to use these settings?

The Materials, .mat files, are to be imported into the materials, by pressing the F4 key and importing them. Now you can assign a physics material to all the objects that are in contact with the ball. The name of the materials will give a clue where to use them.

The Table's physics, the .vpp files, are to be imported in the table's properties by clicking on the "Import physics Set" button (which include also the flipper physics). Change the slope to fit the era of the table. As default the EM the slope is set to 5 and in the SS version the slope is set to 6. But older EM tables may use 4 or less.

The Script

The script parts of my physics consists in two parts, the flipper timer and the ball speed & spin control, which I added to my ball rolling sound routine.

The flipper script is just a change in real time of the EOS speed, which is needed to be able to do some flipper tricks. When a flipper is hold up the EOS Coil is fully charged, the Ampere increases due to flipper can't move or when it pushed back when "On". EOS Coil have more power. When flipper rotate to end, when flipper moves, EOS coil have less Ampere due to flipper can freely move. EOS Coil have less power.

With this script/timer you'll be able to catch the ball when the flipper is moving, due to its weaker EOS, and the flipper will not bounce the it is holdt up, due to its stronger EOS.

There are two variables that must be set when you press a key, so the timer will change the EOS according to when the flippers move up or down. These are LeftFlipperOn and RightFlipperOn. Here it is an example when to use them:

```
Sub SolLFlipper(Enabled)
```

```
  If Enabled Then
```

```
    PlaySoundAt SoundFXDOF("fx_flipperup", 101, DOFOn, DOFFlippers), LeftFlipper
```

```
    LeftFlipper.RotateToEnd
```

```
    LeftFlipperOn = 1
```

```
  Else
```

```
    PlaySoundAt SoundFXDOF("fx_flipperdown", 101, DOFOff, DOFFlippers), LeftFlipper
```

```
    LeftFlipper.RotateToStart
```

```
    LeftFlipperOn = 0
```

```
  End If
```

```
End Sub
```

```
Sub SolRFlipper(Enabled)
```

```
  If Enabled Then
```

```
    PlaySoundAt SoundFXDOF("fx_flipperup", 102, DOFOn, DOFFlippers), RightFlipper
```

```
    RightFlipper.RotateToEnd
```

```
    RightFlipperOn = 1
```

```
  Else
```

```
    PlaySoundAt SoundFXDOF("fx_flipperdown", 102, DOFOff, DOFFlippers), RightFlipper
```

```
    RightFlipper.RotateToStart
```

```
    RightFlipperOn = 0
```

```
  End If
```

```
End Sub
```

And the script itself:

```
*****  
' Real Time Flipper EOS adjustment  
' based on JLouLouLou flipper script  
'   JP's physics Version 2026  
*****
```

```
Dim LeftFlipperOn  
Dim RightFlipperOn
```

```
' EOS Torque when flipper hold up ( EOS Coil is fully charged.  
' Ampere increase due to flipper can't move or when it pushed back when "On". EOS Coil  
have more power )  
Const FullStrokeEOS_Torque = 0.9
```

```
' EOS Torque when flipper rotate to end ( When flipper move, EOS coil have  
' less Ampere due to flipper can freely move. EOS Coil have less power )  
Const LiveStrokeEOS_Torque = 0.2
```

```
LeftFlipper.TimerInterval = 1  
LeftFlipper.TimerEnabled = 1
```

```
Sub LeftFlipper_Timer 'flipper's EOS changer  
  'Left End Of Stroke Routine  
  If LeftFlipperOn = 1 Then  
    If LeftFlipper.CurrentAngle = LeftFlipper.EndAngle then  
      LeftFlipper.EOSTorque = FullStrokeEOS_Torque  
    End If  
  Else  
    LeftFlipper.EOSTorque = LiveStrokeEOS_Torque  
  End If  
  'Right End Of Stroke Routine  
  If RightFlipperOn = 1 Then  
    If RightFlipper.CurrentAngle = RightFlipper.EndAngle Then  
      RightFlipper.EOSTorque = FullStrokeEOS_Torque  
    End If  
  Else  
    RightFlipper.EOSTorque = LiveStrokeEOS_Torque  
  End If  
End Sub
```

The second part of the script, the speed & spin control is a very short script that I included in the rolling sound routine, as it is an operation that is done almost at realtime and on all the balls, the same as the rolling sound, so it was the right place to add it Check later for the full rolling sound routine.

```
' jps ball spin control
BOT(b).AngMomZ = BOT(b).AngMomZ * 0.95

' jps ball speed control 2026
If ABS(BallVel(BOT(b) ) )> maxvel Then
  If BOT(b).VelX <> 0 Then
    speedfactorx = ABS(maxvel / BOT(b).VelX)
    If speedfactorx <1 Then
      BOT(b).VelX = BOT(b).VelX * speedfactorx
      BOT(b).VelY = BOT(b).VelY * speedfactorx
    End If
  End If
  If BOT(b).VelY <> 0 Then
    speedfactory = ABS(maxvel / BOT(b).VelY)
    If speedfactory <1 Then
      BOT(b).VelX = BOT(b).VelX * speedfactory
      BOT(b).VelY = BOT(b).VelY * speedfactory
    End If
  End If
End If
```

Some EXTRA routines

These is my version of the support functions for the sounds.

Note that they are different from other authors. I use the ball speed and some extra some random to change the volume and the pitch on the PlaySoundAt and PlaySoundAtBall so one single sound can sound differently, both according the ball speed and some random. This way with a single sound file I can produce almost infinite variations.

PlaySoundAtBall I use this for the ball hit sounds when the ball hits some objects, walls, ramps, targets. A faster ball will sound louder and in a higher pitch than a slower ball.

PlaySoundAT I use this for the sounds some objects do when they are activated, slingshots, bumpers, flippers, diverters, droptargets, kickers and solenoids in general.

```

*****
'      Supporting Ball & Sound Functions 2026
' includes random pitch in PlaySoundAt and PlaySoundAtBall
*****

```

```

Dim TableWidth, TableHeight

```

```

TableWidth = Table1.width
TableHeight = Table1.height

```

```

Function Vol(ball) ' Calculates the Volume of the sound based on the ball speed
    Vol = Csng(BallVel(ball) ^2 / 2000)
End Function

```

```

Function Pan(ball) ' Calculates the pan for a ball based on the X position on the table.
"table1" is the name of the table
    Dim tmp
    tmp = ball.x * 2 / TableWidth-1
    If tmp> 0 Then
        Pan = Csng(tmp ^10)
    Else
        Pan = Csng(-((- tmp) ^10) )
    End If
End Function

```

```

Function Pitch(ball) ' Calculates the pitch of the sound based on the ball speed
    Pitch = BallVel(ball) * 20
End Function

```

```

Function BallVel(ball) 'Calculates the ball speed
    BallVel = (SQR((ball.VelX ^2) + (ball.VelY ^2) ) )
End Function

```

```

Function AudioFade(ball) 'only on VPX 10.4 and newer
    Dim tmp
    tmp = ball.y * 2 / TableHeight-1
    If tmp> 0 Then
        AudioFade = Csng(tmp ^10)
    Else
        AudioFade = Csng(-((- tmp) ^10) )
    End If
End Function

```

```

Sub PlaySoundAt(soundname, tableobj) 'play sound at X and Y position of an object, mostly
bumpers, flippers and other fast objects
    PlaySound soundname, 0, 1, Pan(tableobj), 0.2, 0, 0, 0, AudioFade(tableobj)

```

End Sub

Sub PlaySoundAtBall(soundname) ' play a collision sound at the ball position, like rubbers, targets, metals, plastics

 PlaySound soundname, 0, Vol(ActiveBall), pan(ActiveBall), 0, Pitch(ActiveBall) * 10, 0, 0, AudioFade(ActiveBall)

End Sub

Function RndNbr(n) 'returns a random number between 1 and n

 Randomize timer

 RndNbr = Int((n * Rnd) + 1)

End Function

Const Pi=3.1415926535

And finally the rolling timer with the ball speed & spin control, which you can see at the end of the sub.

' JP's VPX8 Ball Rolling Sounds 2026

Const tnob = 19 'total number of balls, use a high number so you may test the table without errors.

Const lob = 0 'number of locked balls

ReDim rolling(tnob)

Const maxvel = 48 'max ball velocity. EM: 20-35 SS: 35-50

InitRolling

Sub InitRolling

 Dim i

 For i = 0 to tnob

 rolling(i) = False

 Next

 RollingTimer.Enabled = 1

End Sub

Sub RollingTimer_Timer()

 Dim BOT, b, ballpitch, speedfactorx, speedfactory

 BOT = GetBalls

 ' stop the sound of deleted balls

 For b = UBound(BOT) + 1 to tnob

```

    rolling(b) = False
    StopSound("fx_ballrolling" & b)
Next

' exit the sub if no balls on the table
If UBound(BOT) = lob - 1 Then Exit Sub 'there no extra balls on this table

' play the rolling sound for each ball
For b = lob to UBound(BOT)
    If BallVel(BOT(b)) > 1 Then
        If BOT(b).z < 0 Then
            ballpitch = Pitch(BOT(b)) - 5000 'decrease the pitch under the playfield
        ElseIf BOT(b).z < 30 Then
            ballpitch = Pitch(BOT(b))
        Else
            ballpitch = Pitch(BOT(b)) + 30000 'increase the pitch on a ramp or platform
        End If
        rolling(b) = True
        PlaySound("fx_ballrolling" & b), -1, Vol(BOT(b)), Pan(BOT(b)), 0, ballpitch, 1, 0,
AudioFade(BOT(b))
    Else
        If rolling(b) = True Then
            StopSound("fx_ballrolling" & b)
            rolling(b) = False
        End If
    End If

' dropping sounds
If BOT(b).VelZ < -1 Then
    'from ramp
    If BOT(b).z < 55 and BOT(b).z > 27 Then PlaySound "fx_balldrop", 0, ABS(BOT(b).velz) /
17, Pan(BOT(b)), 0, Pitch(BOT(b)), 1, 0, AudioFade(BOT(b))
    'down a hole
    If BOT(b).z < 10 and BOT(b).z > -10 Then PlaySound "fx_hole_enter", 0,
ABS(BOT(b).velz) / 17, Pan(BOT(b)), 0, Pitch(BOT(b)), 1, 0, AudioFade(BOT(b))
    End If

' jps ball spin control
BOT(b).AngMomZ = BOT(b).AngMomZ * 0.95

' jps ball speed control 2026
If ABS(BallVel(BOT(b))) > maxvel Then
    If BOT(b).VelX <> 0 Then
        speedfactorx = ABS(maxvel / BOT(b).VelX)
        If speedfactorx < 1 Then
            BOT(b).VelX = BOT(b).VelX * speedfactorx
            BOT(b).VelY = BOT(b).VelY * speedfactorx
        End If
    End If
End If

```

```
        End If
    End If
    If BOT(b).VelY <> 0 Then
        speedfactory = ABS(maxvel / BOT(b).VelY)
        If speedfactory <1 Then
            BOT(b).VelX = BOT(b).VelX * speedfactory
            BOT(b).VelY = BOT(b).VelY * speedfactory
        End If
    End If
End If
End If
Next
End Sub
```